

CSE 309: Theory of Automata

Introduction and Finite Automata

Shahid Hussain

shahidhussain@iba.edu.pk

Spring2025

School of Mathematics and Computer Science

Institute of Business Administration

- One of the most abstract courses in entire CS curriculum
- We will see what is computation?
- Applications (finite state machines)
 - Compiler construction
 - String searches/word processor

What is a computer?

- It is too difficult to answer such a question
- We use an idealized computer called **computational model**
- We begin with **finite state machine** or **finite automata**

Computational Model

- These machines compute yes/no questions
- Set of binary strings that end in 0
- Set of binary strings that represent legal Python/C++ program
- Set of binary strings that represent legal Java/C++ program that does not go into infinite loop
- What can you compute and what can't you compute? not just in terms of a programming language rather in a more abstract and mathematical terms e.g., Turing machines

Strings and Languages

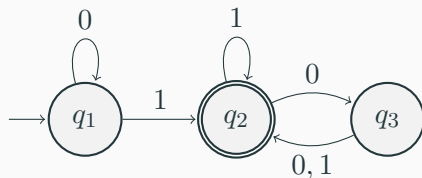
- Strings of characters are fundamental building blocks
- An *alphabet* is the set of such fundamental characters called *symbols*
- We use Σ and Γ to denote sets of alphabets
- A *string over an alphabet* is a finite sequence from the alphabet
- Let $\Sigma_1 = \{0, 1\}$ then 01101 is a string over Σ_1
- If w is a string over Σ , the *length* of w , written as $|w|$, is the number of symbols in w

- The string of length zero is called *empty string* and denoted as ϵ
- If w has length n we can write $w = w_1w_2 \dots w_n$ where each $w_i \in \Sigma$
- The reverse of w , written as w^{rev} , is the string obtained by writing w in opposite order i.e., $w_nw_{n-1} \dots w_1$
- String z is a *substring* of w if z appears consecutively within w , for example 101 is a substring of 0111010

- If we have strings x and y of length m and n , respectively, the *concatenation* of x and y , written as xy , is the string $x_1 \cdots x_m y_1 \cdots y_n$
- x^k means $\overbrace{xx \cdots x}^k$
- The *lexicographic order* of strings is the dictionary order of the strings
- x is called a *prefix* of string y if a string z exists where $xz = y$ and x is *proper prefix* of y if in addition $x \neq y$
- A *language* is a set of strings
- A language is *prefix-free* if no member is a proper prefix of another member

- A finite automaton is a model for computation
- It is extremely limited memory
- Examples include: automatic doors which swing open when they sense some person

Finite Automata



- Call it M_1 , it has three states, q_1 , q_2 , and q_3
- Start or initial state: q_1 (with an incoming arrow)
- Accepting or final state: q_2 (with double circle)
- Arrows show transitions
- Assume the input sequence 1101 to M_1
- It is accepted

Finite Automata

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called **states**,
2. Σ is a finite set called **alphabet**,
3. $\delta : Q \times \Sigma \rightarrow Q$ is a **transition function**,
4. $q_0 \in Q$ is the **start state**, and
5. $F \subseteq Q$ is the **set of accept states** (or **final states**).

Finite Automata

- The finite automaton M_1 :
- $Q = \{q_1, q_2, q_3\}$
- $\Sigma = \{0, 1\}$,
- δ is defined as:

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

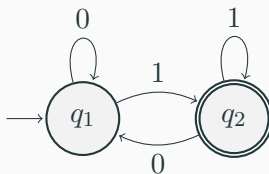
- q_1 is the start state
- $F = \{q_2\}$

Languages of Finite Automata

- If A is the set of all strings that machine M accepts, we say that A is the **language of machine M**
- We write $L(M) = A$ and say M **accepts** A or M **recognizes** A
- A machine may accept several strings but it recognizes only one language
- For the machine M_1 ,

$$A = \{w : w \text{ contains at least one } 1 \text{ and} \\ \text{an even number of } 0\text{'s follow the last } 1\}$$

Example M_2

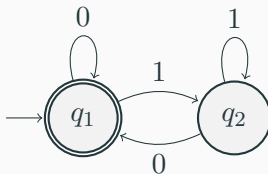


- $M_2 = \{\{q_1, q_2\}, \{0, 1\}, \delta, q_1, \{q_2\}\}$, where δ is defined as

	0	1
q_1	q_1	q_2
q_2	q_1	q_2

- $L(M_2) = \{w : w \text{ ends in a } 1\}$

Example M_3



- $L(M_3) = \{w : w \text{ is the empty string } \epsilon \text{ or ends in a } 0 \}$

Regular Language

A language is called a **regular language** if some finite automaton recognizes it.

- Let A and B be languages. We define regular operations **union**, **concatenation**, and **star** as follows:

- **Union:**

$$A \cup B = \{x : x \in A \text{ or } x \in B\}$$

- **Concatenation:**

$$A \circ B = AB = \{xy : x \in A \text{ and } y \in B\}$$

- **Star:**

$$A^* = \{x_1x_2 \dots x_k : k \geq 0 \text{ and each } x_i \in A\}$$

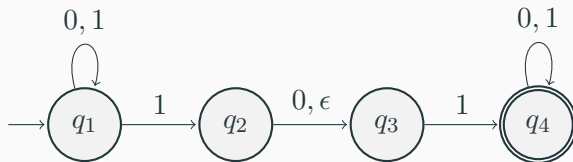
Theorem 1

The class of regular languages is closed under:

1. *the union operation*
2. *the concatenation operation*

Nondeterminism

- Nondeterminism is a useful concept
- In our automata every step of computation follows in a unique way from the preceding step
- Given a state and the next input symbol we know exactly what will be the next state of machine
- That is, it can be determined
- In **nondeterministic machines** several choices may exist for the next state at any point



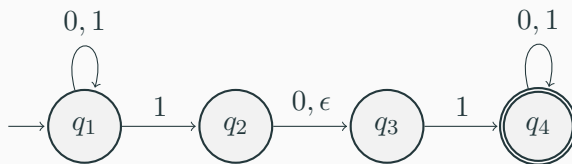
Nondeterministic Finite Automata

Nondeterministic Finite Automata

A **nondeterministic finite automata** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta : Q \times \Sigma_{\epsilon} \rightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

Example NFA



1. $Q = \{q_1, q_2, q_3, q_4\}$
2. $\Sigma = \{0, 1\}$
3. Transition function δ :

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

4. q_1 is the start state, and

- Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA
- $w = y_1 y_2 \cdots y_m$ over some alphabet Σ_ϵ
- $r_0, r_1, \dots, r_m$ exists in Q such that:
 1. $r_0 = q_0$,
 2. $r_{i+1} \in \delta(r_i, y_{i+1})$, for $i = 0, \dots, m-1$, and
 3. $r_m \in F$
- Then, we say N **accepts** w

Theorem 2

Every nondeterministic finite automaton has an equivalent deterministic finite automaton.

- If we could show for a language A recognized by an NFA can also be recognized by a DFA then we are done with the proof.
- So, let's convert an NFA into a DFA.

Proof: assuming no ϵ transitions

Proof.

- $N = (Q, \Sigma, \delta, q_0, F)$ be the NFA recognizing some language A
- DFA $M = (Q', \Sigma, \delta', q'_0, F')$ recognizing A
- $Q' = \mathcal{P}(Q)$
- For $R \in Q'$ and $a \in \Sigma$, let $\delta'(R, a) = \{q \in Q : q \in \delta(r, a) \text{ for some } r \in R\}$ or

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).$$

- $q'_0 = \{q_0\}$
- $F' = \{R \in Q' : R \text{ contains an accept state of } N\}$



Proof.

- For any state R of M , we define $E(R)$ to be the collection of states that can be reached from members of R by going only along ϵ arrows including the members of R themselves, for $R \subseteq Q$

$$E(R) = \{q : q \text{ can be reached from } R \text{ traversing } \epsilon \text{ arrows}\}$$

- The new transition function can be written as:

$$\delta'(R, a) = \{q \in Q : q \in E(\delta(r, a)) \text{ for some } r \in R\}.$$

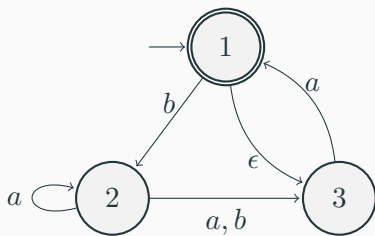
- The new start state of M would be $E(\{q_0\})$



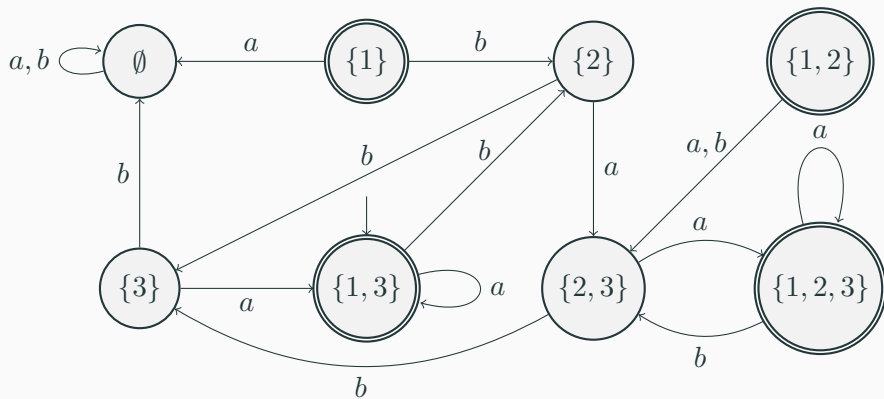
Corollary 3

A language is regular if and only if some nondeterministic finite automaton recognizes it.

Example: NFA to DFA



Example: NFA to DFA



Theorem 4

The class of regular languages is closed under the union operation.

Theorem 5

The class of regular languages is closed under the concatenation operation.

Theorem 6

The class of regular languages is closed under the star operation.

Regular Expression

Say that R is a **regular expression** if R is

1. a for some a in the alphabet Σ ,
2. ϵ ,
3. $\emptyset$,
4. $(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions,
5. $(R_1 \circ R_2)$, where R_1 and R_2 are regular expressions,
6. (R_1^*) , where R_1 is a regular expression.

Examples

- $0^*10^* = \{w : w \text{ contains a single } 1\}$
- $\Sigma^*1\Sigma^* = \{w : w \text{ has at least one } 1\}$
- $(\Sigma\Sigma)^* = \{w : w \text{ is a string of even length}\}$
- $(0 \cup \epsilon)1^* = 01^* \cup 1^*$
- $1^*\emptyset = \emptyset$
- $\emptyset^* = \{\epsilon\}$
- $R \cup \emptyset = R$
- $R \circ \epsilon = R$

- Let R be any regular expression:
- $R \cup \emptyset = R$ and $R \circ \epsilon = R$
- $R \cup \epsilon$ may not equal R e.g., if $R = 0$ then $L(R) = \{0\}$ but $L(R \cup \epsilon) = \{0, \epsilon\}$
- $R \circ \emptyset$ may not equal R e.g., if $R = 0$ then $L(R) = \{0\}$ but $L(R \circ \emptyset) = \emptyset$

Regular Expressions and Regular Languages

Theorem 7

A language is regular if and only if some regular expression describes it.

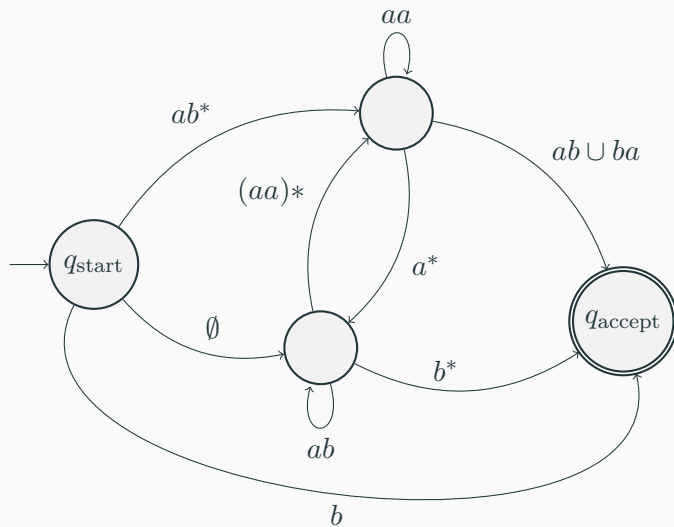
Lemma 8

If a language is described by a regular expression, then it is regular.

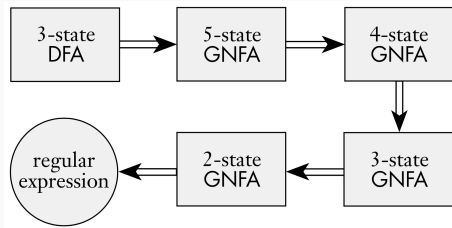
Lemma 9

If a language is regular, then it is described by a regular expression.

Example of GNFA



DFA to Regular Expression using GNFA



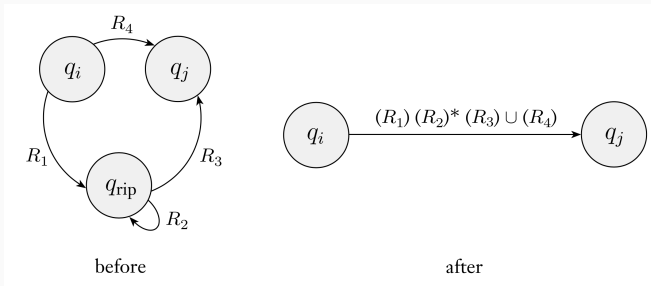
Generalized Nondeterministic Finite Automata

GNFA

A **generalized nondeterministic finite automaton** is a 5-tuple, $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$, where

1. Q is the finite set of states,
2. Σ is the input alphabet,
3. $\delta : (Q \setminus \{q_{\text{accept}}\}) \times (Q \setminus \{q_{\text{start}}\}) \rightarrow \mathcal{R}$ is the transition function,
4. q_{start} is the start state, and
5. q_{accept} is the accept state.

DFA to Regular Expression using GNFA



Generalized Nondeterministic Finite Automata

A GNFA accepts a string $w \in \Sigma^*$ if $w = w_1w_2 \cdots w_k$, where each w_i is in Σ^* and a sequence of states $q_0, q_1, \dots, q_k$ exists such that

1. $q_0 = q_{\text{start}}$ is the start state,
2. $q_k = q_{\text{accept}}$ is the accept state,
3. for each i we have $w_i \in L(R_i)$, where $R_i = \delta(q_{i-1}, q_i)$, in other words, R_i is the expression on the arrow from q_{i-1} to q_i .

Converting DFA to GNFA

CONVERT(G):

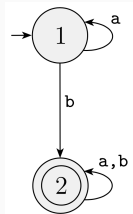
1. Let k be the number of states in G
2. If $k = 2$, then G must consists of a start state, an accept state, and a single arrow connecting them labeled with R.E. R
3. If $k > 2$, we select any state $q_{\text{rip}} \in Q \setminus \{q_{\text{start}}, q_{\text{accept}}\}$ and let G' be the GNFA $(Q', \Sigma, \delta', q_{\text{start}}, q_{\text{accept}})$, where $Q' = Q \setminus \{q_{\text{rip}}\}$, and for any $q_i \in Q' \setminus \{q_{\text{accept}}\}$ and any $q_j \in Q' \setminus \{q_{\text{start}}\}$ let

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4),$$

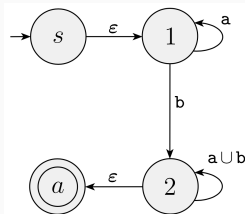
for $R_1 = \delta(q_i, q_{\text{rip}})$, $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$, $R_3 = \delta(q_{\text{rip}}, q_j)$, and $R_4 = \delta(q_i, q_j)$

4. Compute CONVERT(G') and return this value.

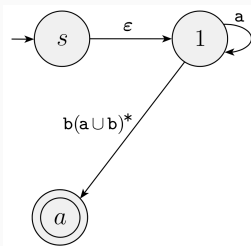
Example: DFA to GNFA



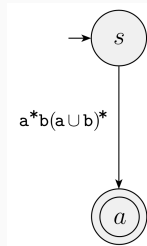
(a)



(b)

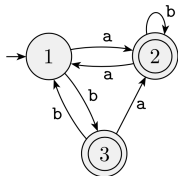


(c)

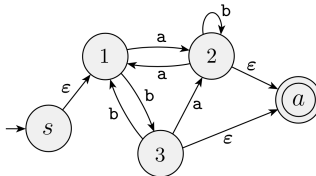


(d)

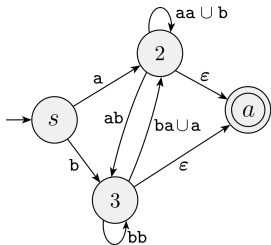
Example: DFA to GNFA



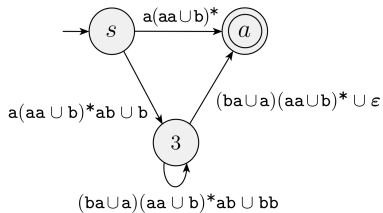
(a)



(b)



(c)



(d)

Example: DFA to GNFA



$(a(aa \cup b)^*ab \cup b)((ba \cup a)(aa \cup b)^*ab \cup bb)^*((ba \cup a)(aa \cup b)^* \cup \varepsilon) \cup a(aa \cup b)^*$

(e)

Nonregular Language

- Not all languages are regular for example

$$L = \{0^n 1^n : n \geq 0\}$$

- Now consider two languages over $\Sigma = \{0, 1\}$, are they regular?

$$C = \{w : w \text{ has an equal number of 0s and 1s}\},$$

$$D = \{w : w \text{ has an equal number of 01s and 10s as substrings}\}$$

- C is not regular but D is :)

Pumping Lemma for Regular Languages

Lemma 10

If A is a regular language, then there is a number p (the pumping length) where, if s is any string in A of length at least p , then s may be divided into three pieces, $s = xyz$, satisfying the following condition:

1. *for each $i \geq 0$, $xy^iz \in A$,*
2. *$|y| > 0$, and*
3. *$|xy| \leq p$.*

Proof for Pumping Lemma

- Let $M = (Q, \Sigma, \delta, q_q, F)$ be a DFA recognizing A and p the number of states of M
- Let $s = s_1 s_2 \dots, s_n$ be a string in A of length n , $n \geq p$
- Let $r_1, \dots, r_{n+1}$ be the sequence of states that M enters while processing s , so $r_{i+1} = \delta(r_i, s_i)$ for $1 \leq i \leq n$, this sequence has length $n + 1$ which is at least $p + 1$
- Among the first $p + 1$ elements in the sequence, two must be the same state (by pigeonhole principle)
- Let first of them be r_j and second r_l , r_l occurs among the first $p + 1$ places in a sequence starting at r_1 , we have $l \leq p + 1$
- Let $x = s_1 \dots s_{j-1}$, $y = s_j \dots s_{l-1}$, and $z = s_l \dots, s_n$
- x takes M from r_1 to r_j , y takes M from r_j to r_j , and z takes M from r_j to r_{n+1} (accepting state)
- M must accept $xy^i z$ for $i \geq 0$ and since $j \neq l$ so $|y| > 0$ and $l \leq p + 1$ for

Using PL to Prove Languages Nonregular

- Let B be the language $\{0^n 1^n : n \geq 0\}$
- Assume B is regular and p be the pumping length given by the pumping lemma
- Let $s = 0^p 1^p$ so $s \in B$ and $|s| > p$ therefore $s = xyz$
- For $i \geq 0$, $xy^i z$ must be in B
 1. y consists only of 0's so $xyyz$ has more 0's than 1's
 2. y consists only of 1, same argument
 3. y consists of both 0's and 1's so $xyyz$ may have equal number of 0's and 1's but they will be out of order
- Hence, B is not regular

Using PL to Prove Languages Nonregular

- Let $C = \{w : w \text{ has an equal number of 0s and 1s}\}$
- Assume C is regular and p be the pumping length
- Let $s = 0^p 1^p$, $s \in C$ and $|s| > p$ therefore $s = xyz$
- Now $|xy| \leq p$ so $s = 0^p 1^p$ cannot be pumped
- If $|xy| \leq p$ then y must consist only of 0's so $xyyz \notin C$
- Alternately, since B is nonregular, if C were regular $C \cap 0^* 1^*$ also would be regular
- But $C \cap 0^* 1^* = B$ and we know B is not regular

Using PL to Prove Languages Nonregular

- Show that $F = \{ww : w \in \{0,1\}^*\}$ is not regular
- Show that $D = \{1^{n^2} : n \geq 0\}$ is not regular. D is the language of all strings of 1s whose length is a perfect square
- Show that $E = \{0^i 1^j : i > j\}$ is not regular